

FRAGEN

1. Welche Teile des Programms müssen gegenüber parallelem Zugriff geschützt werden?

Zuerst muss die Nummerierung der Zielordner geschützt werden. Jeder Aufruf von `crawl(uri)` erhält einen eigenen Unterordner mit einer fortlaufenden Nummer. Da mehrere Threads diese Methode gleichzeitig aufrufen können, darf die Nummerierung nicht doppelt vergeben werden.

Ebenfalls kritisch ist die Zustandsverfolgung des Crawlers. Der Crawler muss jederzeit wissen, wie viele Webseiten-Scans warten oder laufen und wie viele Bilddownloads warten oder laufen. Diese Werte werden von unterschiedlichen Threads verändert: vom aufrufenden Thread beim Einreichen neuer Arbeit und von den Worker-Threads beim Starten und Beenden ihrer Aufgaben.

Auch `isIdle()` greift auf gemeinsam genutzte Daten zu. Die Methode darf nur dann `true` zurückgeben, wenn weder Webseitenanalysen noch Bilddownloads offen sind. Deshalb müssen die zugrunde liegenden Zähler sicher aktualisiert und gelesen werden können.

Ein weiterer gemeinsamer Bereich ist die Vergabe der Dateinamen. Wenn mehrere Bilder im selben Zielordner denselben Namen haben, muss eindeutig entschieden werden, welche Datei den Originalnamen behält und welche Datei einen Suffix wie `_2` bekommt. Diese Entscheidung darf bei parallelen Downloads nicht zufällig oder mehrfach getroffen werden.

Die Warteschlangen der Aufgaben müssen ebenfalls thread-safe sein. Diese Struktur wird jedoch nicht selbst implementiert, sondern durch `ExecutorService` bereitgestellt. Die Java-Bibliothek übernimmt hier das sichere Einreihen und Abarbeiten wartender Aufgaben.

2. Wie stellen Sie den Schutz gegenüber dem parallelen Zugriff an den genannten Stellen sicher? Welche Alternativen hätte es gegeben? Warum haben Sie sich für diese Variante des Schutzes entschieden?

Für die Ordernummern und die Aufgabenzähler verwendet das Programm `AtomicInteger`. Das ist an diesen Stellen passend, weil hauptsächlich einfache Operationen wie Erhöhen, Verringern und Lesen benötigt werden. Ein großer gemeinsamer Lock wäre hier unnötig schwerfällig und würde die parallele Verarbeitung stärker einschränken.

Die Begrenzung der Parallelität erfolgt über zwei feste `ExecutorService`-Threadpools. Ein Pool verarbeitet ausschließlich Webseiten-Scans, der andere Pool ausschließlich Bilddownloads. Die jeweilige maximale Threadanzahl kommt direkt aus der Konfiguration. Wenn alle Worker eines Pools beschäftigt sind, bleiben weitere Aufgaben in der internen Warteschlange des Executors, bis wieder ein Platz frei wird.

Die Dateinamenvergabe ist im `FileNameResolver` mit einer `synchronized`-Methode geschützt. Der kritische Bereich ist bewusst klein gehalten. Innerhalb dieses Bereichs wird nur der nächste freie Name berechnet und reserviert. HTTP-Downloads und Dateischreiboperationen laufen außerhalb dieser Synchronisierung. Dadurch blockieren sich parallele Downloads nicht unnötig gegenseitig.

Eine Alternative wäre gewesen, größere Teile von `ImageCrawler` vollständig zu synchronisieren. Das wäre zwar einfach umzusetzen, würde aber zu viel unabhängige Arbeit blockieren. Auch ein `ReentrantLock` wäre möglich gewesen. Für diese Implementierung

bringt er jedoch keinen echten Vorteil gegenüber `AtomicInteger` und einem kleinen `synchronized`-Block.

Man könnte die Parallelität außerdem mit `Semaphore` oder eigenen Worker-Queues begrenzen. Für diese Aufgabe ist `ExecutorService` aber die direktere Lösung, weil Java damit das Einreihen, Warten und Abarbeiten von Aufgaben bereits zuverlässig bereitstellt. Die Implementierung bleibt dadurch übersichtlich und konzentriert sich auf die eigentliche Crawler-Logik.

3. Nennen Sie die Objekte und Datenstrukturen, die pro Thread zusätzlich im Arbeitsspeicher Heap oder Stack vorgehalten werden müssen. JVM-interne Details zum erforderlichen Speicher für einen Thread müssen nicht behandelt werden.

Pro Worker-Thread entstehen Stack Frames für die Methoden, die gerade ausgeführt werden. Bei einem Webseiten-Scan sind das zum Beispiel Aufrufe in `runWebsiteTask` und `WebsiteAnalyzer.analyze`. Dazu kommen lokale Variablen wie die Webseiten-URI, der Zielordner, der HTTP-Request und die HTTP-Response.

Bei der Webseitenanalyse wird der HTML-Text im Speicher gehalten. Jsoup erzeugt daraus ein `Document`, aus dem die `<img>`-Elemente gelesen werden. Die extrahierten Bildadressen liegen anschließend als Liste von `URI`-Objekten vor, bis daraus Download-Aufgaben erzeugt wurden.

Bei einem Bilddownload hält der Thread ebenfalls lokale HTTP-Objekte, also einen `HttpRequest` und eine `HttpResponse<byte[]>`. Die heruntergeladenen Bilddaten liegen als Byte-Array im Speicher, bevor sie in eine Datei geschrieben werden. Dazu kommen lokale Variablen für den Zielpfad und den Dateinamen.

Im Heap liegen zusätzlich die Aufgabenobjekte, die in den Executor-Queues warten oder gerade ausgeführt werden. Dazu gehören die `Runnable`-Instanzen und die `ImageReference`-Objekte, die jeweils eine Bild-URI und den passenden Zielordner beschreiben.

Wenn Fehler auftreten, können kurzzeitig Exception-Objekte entstehen. Diese werden abgefangen, damit ein einzelner defekter Link oder Download nicht den gesamten Crawler beendet.